

CLOUD INFRASTRUCTURE AND FRAMEWORK REPORT

An AWS CloudFormation Report

Development Team Project by COM January 2026 Group 2

Submission Date: 8 March 2026

Overview

AWS CloudFormation is a service provided by Amazon Web Services (AWS) that provisions, configures, and manages your AWS resources (AWS, 2025). Its core feature is the use of templates to model infrastructure architecture (Amazon Web Services, 2015). These templates are YAML or JSON files which serve as blueprints for the configuration of all AWS resources, which refer to the elements created via AWS CloudFormation (*How CloudFormation works - AWS CloudFormation*, 2024).

For the management of templates, AWS CloudFormation allows for version control of templates and the use of stacks, which are ways of grouping your templates and resources based on the same application or other custom logic (*How CloudFormation works - AWS CloudFormation*, 2024). Additionally, another key feature of AWS CloudFormation is its incremental nature in implementing the uploaded templates. This is done through what is known as 'Change sets', which detect the difference in current and uploaded templates to only implement the difference between the two (*How CloudFormation works - AWS CloudFormation*, 2025).

Service Model Analysis

AWS CloudFormation supports cloud service models like IaaS, PaaS, and SaaS by acting as an Infrastructure as Code (IaC) solution. It helps replicate infrastructure across multiple instances in an automated way, hence reducing manual workload and human error. Furthermore, any changes in infrastructure can be managed through a change management process backed by code reviews, again reducing human error (*CloudFormation best practices - AWS CloudFormation*, 2026).

For IaaS, AWS CloudFormation templates can be used to provision the building blocks of an environment, like the Amazon Simple Storage Service, also known as Amazon S3 (*Amazon Simple Storage Service (Amazon S3) - AWS CloudFormation*, 2026). With the PaaS service model, AWS CloudFormation can be used to deploy managed application environments like AWS Elastic Beanstalk (AWS, 2025). For SaaS, AWS CloudFormation allows users to configure AWS IAM, which can be integrated with applications. For example, organisations can automate the provisioning of separate environments for tenants by using AWS CloudFormation templates to manage the lifecycle of resources like Amazon S3, AWS Elastic Beanstalk and AWS IAM.

Deployment Types

CloudFormation is best suited for public cloud environments. Since AWS owns and manages the underlying infrastructure, CloudFormation can provision resources on demand without any additional configuration. This makes it the most straightforward deployment option. The CloudFormation templates define everything upfront, like EC2 instances, RDS databases, IAM roles, and the same templates can be deployed repeatedly with identical results, removing the guesswork that comes with manual configuration (Cotcharat, 2022).

However, a private cloud deployment is better suited for some organisations. For instance, healthcare providers and financial institutions often work under compliance frameworks like GDPR or ISO 27001 that restrict where data can physically reside, due to the involvement of sensitive data (Jansson, 2021). To ensure compliance, AWS Outposts can be used to bring AWS hardware into an organisation's own data centre, so CloudFormation templates can be used without the organisation's data ever leaving a controlled environment.

On the other hand, hybrid deployments tend to emerge gradually rather than being planned from the start. Direct Connect handles the connectivity between on-premises and AWS, and since CloudFormation uses the same template structure across both environments, teams can manage their entire infrastructure through one consistent toolset rather than maintaining separate provisioning processes for each side (Jansson, 2021).

Infrastructure Design

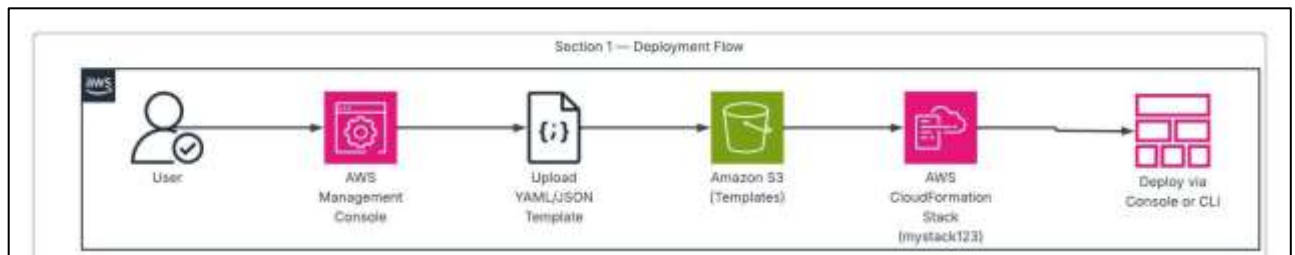


Diagram 1: Deployment Flow

As shown in Diagram 1 above, building infrastructure with CloudFormation starts with writing a YAML or JSON template, which can be uploaded directly or stored in S3 for larger deployments. The templates can then be deployed as a stack via the console or CLI.

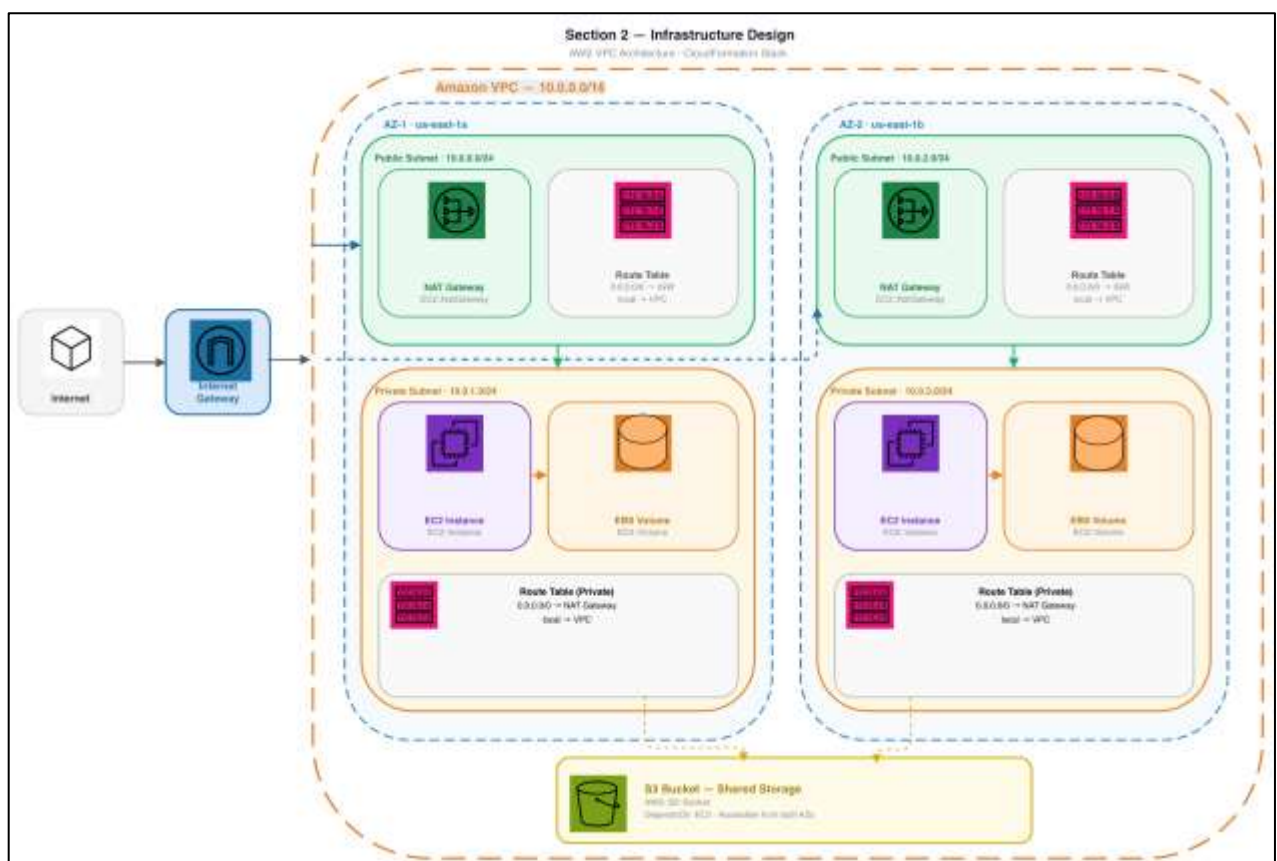


Diagram 2: Infrastructure Design

As can be seen in Diagram 2 above, the network is built around a VPC (10.0.0.0/16). The VPC is further divided into public and private subnets across two availability zones (Cotcharat, 2022). Public subnets handle external-facing traffic through an internet gateway, while private subnets keep backend workloads isolated. Splitting across two availability zones ensures availability if one zone goes down.

Within the private subnet, the compute element is represented by the EC2 resource blocks specifying the instance type and target subnet. Moreover, storage is declared either on an instance-level as an EBS volume or across the two availability zones as an S3 bucket. Furthermore, CloudFormation's DependsOn attribute explicitly defines resource dependencies to ensure that infrastructure components are created in the required sequence. For instance, it ensures that the EC2 instance is created before the S3 storage by enforcing a specific resource creation sequence.

Case Study

World Wide Technology (WWT) provides consulting, engineering, and supply chain solutions using advanced technology (World Wide Technology, no date b). They utilize AWS CloudFormation to develop multistack deployments. These deployments follow a complex order of operations using Custom Lambda Resources, which are user-defined serverless compute functions that execute specialized deployment logic. This approach leverages CloudFormation's deep integration with AWS, pausing deployments until receiving a success signal from Lambda (World Wide Technology, no date a). WWT prefers this over Nested Stacks (sub-stacks created within a parent stack) to better manage complex dependencies and updates on the template project.

WWT also deploys services adhering to the Least Privilege principle, ensuring principals possess only the minimum access needed for their tasks (Nieles, Dempsey and Pillitteri, 2017). This illustrates how CloudFormation supports security and compliance (Karen Tovmasyan, 2023). For example, WWT utilizes Security Groups (virtual network firewalls) to restrict resource communication, and NAT Gateways to reduce external exposure. Furthermore, code parameters were used to define important information about the deployment. By unsetting parameters, we can prevent the creation of unnecessary resources.

The primary benefits of AWS CloudFormation lie in its operational efficiency, cost-saving flexibility, and robust state management. By utilizing this IaC approach, WWT successfully reduces deployment time and minimizes errors while maintaining working standard configurations. CloudFormation's flexibility is further enhanced through conditional code parameters, allowing administrators to avoid the creation of unnecessary infrastructure and lower deployment costs. Finally, CloudFormation

maintains a centralized state of all deployed services. While stateful resources like populated S3 buckets naturally resist deletion to prevent data loss, CloudFormation can seamlessly trigger custom cleanup logic during stack removal. This automated teardown completely removes all requested resources, mitigating human error and preventing financial leakage from orphaned cloud assets.

Conclusion

In short, AWS CloudFormation facilitates the deployment of different cloud service models (IaaS, PaaS, SaaS). It provides the flexibility to be deployed over public, private, or hybrid environments. Moreover, cloud architects can leverage CloudFormation to design cloud infrastructure using key components (compute, storage, networking). This was seen in the WWT use case where real-world benefits like organisational efficiency and cost savings were demonstrated through the use of AWS CloudFormation.

Bibliography

Amazon Simple Storage Service (Amazon S3) - AWS CloudFormation (2026)

Amazon.com. Available at:

https://docs.aws.amazon.com/AWSCloudFormation/latest/TemplateReference/AWS_S3.html (Accessed: 3 March 2026).

Amazon Web Services (2015) *Introduction to AWS CloudFormation*, YouTube.

Available at: https://www.youtube.com/watch?v=Omppm_YUG2g (Accessed: 22 February 2026).

Amazon Web Services (2024) *Amazon EC2*, Amazon Web Services, Inc. Available at: <https://aws.amazon.com/ec2/>.

AWS (2025) *AWS Elastic Beanstalk – Deploy Web Applications*, Amazon Web Services, Inc. Available at: <https://aws.amazon.com/elasticbeanstalk/>.

AWS (2025) *What is AWS CloudFormation? - AWS CloudFormation*, docs.aws.amazon.com. Available at: <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/Welcome.html>.

CloudFormation best practices - AWS CloudFormation (2026) Amazon.com.

Available at:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/best-practices.html#code> (Accessed: 3 March 2026).

Cotcharat, S. (2022) *Infrastructure as Code Tools Comparison on AWS Cloud Environment*. Master of Engineering Thematic Paper. Dhurakij Pundit University.

Available at: <https://libdoc.dpu.ac.th/thesis/Songwut.Cotc.pdf> (Accessed: 22 February 2026).

How CloudFormation works - AWS CloudFormation (2024) Amazon.com. Available at:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/cloudformation-overview.html>.

How CloudFormation works - AWS CloudFormation (2025) Amazon.com. Available at:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/cloudformation-overview.html#cfn-concepts-templates>.

Jansson, I. (2021) *Continuous Compliance Automation in AWS Cloud Environment*.

Master of Science Thesis in Computer Engineering. Åbo Akademi University.

Available at:

https://www.doria.fi/bitstream/handle/10024/181213/jansson_isak.pdf?sequence=2&isAllowed=y (Accessed: 21 February 2026).

Karen Tovmasyan (2023) *Mastering AWS CloudFormation - Second Edition*. Packt Publishing.

Nieles, M., Dempsey, K. and Pillitteri, V.Y. (2017) *An introduction to information security*. NIST Special Publication 800-12 Revision 1. Gaithersburg: National Institute of Standards and Technology. Available at:
<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-12r1.pdf>
(Accessed: 25 February 2026).

Pradip Kotkar (2024) *Choosing Multi-Tenant SaaS Architectures: Silo, Pool, or Bridge Models: A DevOps Perspective*, Medium. nonstopio. Available at:
<https://blog.nonstopio.com/choosing-multi-tenant-saas-architectures-silo-pool-or-bridge-models-a-devops-perspective-fff92216672c> (Accessed: 22 February 2026).

World Wide Technology (WWT) (no date a) *Enabling rapid multi-region deployments with AWS CloudFormation and Lambda*. Available at:
<https://www.wwt.com/case-study/enabling-rapid-multi-region-deployments-with-aws-cloudformation-and-lambda> (Accessed: 20 February 2026).

World Wide Technology (WWT) (no date b) *Who we are - overview - WWT*.

Available at: <https://www.wwt.com/corporate/who-we-are/overview#at-a-glance>

(Accessed: 20 February 2026).